

Command line interface: Introduction to basic command line and file navigation

- Command line interface: Introduction to basic command line and file navigation
 - Welcome and Learning Objectives
 - Copyright information
 - Link to videos
 - Course materials
 - 1. Computer and file basics
 - Navigation
 - `ls` (list files)
 - `man` (manual)
 - `cd` (change directory)
 - Terminal shortcuts save time!
 - file paths (`.`, `..`, `/`)
 - wild cards `*`
 - 2. Creating and editing folders and files
 - `mkdir` (create a folder)
 - `cat` 🐱 (concatenate and print)
 - `touch` (create a file)
 - `>>` and `>` (redirection)
 - `mv` (move and rename)
 - `cp` (copy)
 - `rm` (remove file/directory)
 - `head` and `tail`
 - 3. Helpful commands, introduction to programming structures
 - `clear`, `history`
 - `echo` (print a variable)
 - A short introduction to variables
 - For loops
 - Recaps/cheat sheets
 - Part 1 recap
 - Part 2 recap
 - Part 3 recap
 - Extras: Problem solving and finding answers online
 - Safety risks while looking for answers online
 - Final notes
 - Extra materials for the extra-material-inclined
 - Copyright information: redux

Welcome and Learning Objectives

I am dedicated to equity-focused teaching. I will do my best to make the material accessible and transparent to all participants, regardless of background. This course material should ultimately leave participants feeling confident about their ability to navigate around their file system and execute basic shell

programs. Also, students should leave familiar with vocabulary and concepts which allow them to ask questions and find answers using Google or ChatGPT.

Copyright information

This course material, as well as the accompanying slides and videos are all © Alexa S. Tyszka 2025. These materials may not be distributed or uploaded to any third-party websites without permission of the author.

Link to videos

[The videos can be found here](put a link here lol)

Course materials

1. Computer and file basics

This section will cover some of the basics of interacting with your computer through the command line.

Navigation

The most common way to interact with your computer is through a GUI (Graphical User Interface). Actions like clicking and dragging a file, or renaming a file, or creating a folder, are done through menus and buttons that easily display what their functions are. Underneath the GUI, like a skeleton, the computer is told to perform a cascade of actions that result in the file being moved, or the folder name changing, for example. When we use the command line interface (CLI), we get closer to operating in the same language as the computer, without as much visual output, and we gain the power to interact with our computers through scripts and complex actions.

Since many of us are familiar with the basics of moving files around a desktop and creating folders, this is one of the first exercises that we can do using the CLI.

Depending on your familiarity with your operating system and its file explorer, this might feel very familiar. One way to visualize what is going on is to open files in your file explorer at the same time as you edit them using the CLI.

Ready? Now let's open your terminal window. You'll see some kind of a prompt. Note: my prompt is `>>`. It'll appear when I copy and paste my output. You do not need to type this into your input since it just lets me know where I started typing. You can change yours if you'd like, but we can get to that later.

ls (list files)

Let's run this command first. Type this into the window:

```
ls
```

You'll see a list of the folders and files you have in your root directory!

```

~ » ls
AlDente          Public
Applications     Trimmomatic
BIOS430          Zotero
Corset           cv-corpus-7.0-2021-07-21
Desktop          gadm
Documents        iqtree-1.6.12-MacOSX
Downloads        kallisto
Dropbox          kanboard
(etc...)

```

We can list files from inside each of these folders, too. Try one of the files you see:

```
ls Desktop
```

To access a folder inside a folder, you can use `/`. From what you see above, show the file listing of a folder of your choice, or from another place if your desktop is folder-less.

```
ls Desktop/sra
```

Different options exist for using `ls`. When you add an option to a command, you will use `-`. The most basic options will have letters that may have something to do with the type of option. Sometimes, they might not make much sense. They will sometimes have a longer form, which will work the same. I am constantly using cheat sheets or taking my own notes on commands and their options, and Google is definitely your friend here.

Here are some useful options for `ls`. Try them out!

Option	What it does
<code>-l</code>	use a long listing format
<code>-a</code>	do not ignore entries starting with <code>.</code> (show otherwise hidden files)
<code>-m</code>	fill width with a comma separated list of entries
<code>-r</code>	Reverse order while sorting.
<code>-t</code>	sort by time, newest first
<code>-o</code>	Use long listing format without group info.
<code>-x</code>	list entries by lines instead of by columns
<code>-1</code>	list one file per line

```
<!--
```

You can combine options, too, by chaining together letters. You cannot chain long forms. Some options depend on other options, so must be used in tandem.

Full command	What it does
<code>ls -lah</code>	Displays the <code>-l</code> long form, <code>-a</code> all files, in a <code>-h</code> human-readable form
<code>ls -ltrah</code>	Displays files in <code>-l</code> long form, <code>-t</code> time-sorted, <code>-r</code> reverse sorted (so the most recent is at the bottom where you can see it!), <code>-a</code> all files, in a <code>-h</code> human-readable form.
<code>ls -oh</code>	Displays <code>-o</code> long format with no group information <code>-h</code> in human-readable sizes.

[!TIP] Note also that the order of your options doesn't matter. `ls -ltrah` is the same as `ls -latrh`.

man (manual)

One way to learn more about a command is by using the `man` command! You can scroll through the options available for a certain command. The options will range from interesting to entirely obscure. Don't worry if they don't make sense to you.

```
man ls
```

Click `Q` on your keyboard when you're ready to exit the manual page.

cd (change directory)

Thus far, we have been listing files starting from the same directory. In order to actually move around the system, we need to use a different command.

Locate the directory with the course files. It's titled `botany-2025-shell`.

Now run

```
ls -lah
cd examplefiles
ls -lah
```

[!TIP] When using the terminal, you can press `Up/Down` on your keyboard to access previous commands. Another handy shortcut is using `Tab` to complete file paths. When I type `cd`, I also usually hit `Tab` to see the folders that I can move into. When you're presented with multiple folders after pressing `Tab`, you can use the `Up/Down` arrows to select a folder.

Terminal shortcuts save time!

If you accidentally use `cd examplefiles` instead of `cd examplefiles/`, you will usually end up in the right place. But please note that the trailing `/` indicates that this is a directory. `Tab` autocomplete should always include the trailing `/`. By using the `<kbd>Tab</kbd>` autocomplete shortcut, you can avoid some typo-induced frustration.

At this point you may have also noticed that you cannot use the cursor to select a spot to edit your input, so it might be frustrating to `<kbd>DEL</kbd>` all the way back to where you want to edit. You are essentially working on a screen where there is one "insertion point" which you control using the keyboard rather than the mouse. I'm including a few time-saving terminal keyboard shortcuts here that I use all the time.

Shortcut	Action
<code>Ctrl+U</code>	Delete line in terminal
<code>Ctrl+C</code>	Cancel command (this is your emergency 'stop' lever!)
<code>Ctrl+R</code>	R everse search/browse past commands
<code>Ctrl+L</code>	c lear the screen
<code>Ctrl+A</code>	Move the insertion point to the st A rt of the line
<code>Ctrl+E</code>	Move the insertion point to the E nd of the line
<code>Ctrl+W</code>	Delete content to the beginning of the last W ord
<code><!--</code>	<code>Ctrl+C</code>

file paths (`.`, `..`, `/`)

Let's see what's in the folder `examplefiles/alexas_section`

```
cd alexas_section/
ls -lah
```

[!TIP] Notice how the sorting of folders proceeds differently than we would sort them - it is technically alphabetical, but not sequential.

To enter one of these folders, we can use:

```
cd 11/
ls -lah
```

Now that we are here, how can we get back to the main folder? When calling `ls -lah`, you should see a screen like this:

```
ls -lah
total 0
```

```
drwxr-xr-x  3 alexa  staff   96B Feb  6 10:23 .
drwxr-xr-x 16 alexa  staff  512B Feb  6 10:23 ..
-rw-r--r--  1 alexa  staff    0B Feb  6 10:23 11.txt
```

The `.` and `..` represent the current directory and the parent directory, respectively. These are file paths we can call with `cd` and `ls` just like we used folders.

Relative path	Call	Explanation
<code>.</code>	<code>ls .</code> OR <code>cd .</code>	Calls current directory, so <code>cd</code> and <code>ls</code> just show you or take you to the current directory. Note that this is different from <code>cd</code> as a standalone call, which will take you back to your home/root directory.
<code>..</code>	<code>ls ..</code> OR <code>cd ..</code>	
<code>~</code>	<code>ls</code> <code>~/(filepath)</code> OR <code>cd</code> <code>~/(filepath)</code>	<code>~</code> is your root directory , which means that this is a method for getting an absolute path. Since your root directory should retain the same file structure, you can call full file paths from the root directory without worrying about your current location.

[!TIP] **The difference between absolute and relative file paths.** An absolute file path looks like this: `~/Documents/GitHub/botany-2025-shell/examplefiles/alexas_section/11/11.txt`

but if I have navigated to

`~/Documents/GitHub/botany-2025-shell/examplefiles/`

already, I can just find this file with

`ls ./11/11.txt`

You can also call

`ls /11/11.txt`

The difference between absolute and relative file paths will matter more later, when you want to be sure of the locations of your files. To maintain consistency in my scripts, I try to use **absolute paths** so I can remember where exactly I ran an analysis. For simple, quick movement around the filesystem, **relative paths** work great. For now, let's use relative paths and I will try to keep things consistent. Locate your github folder, and if you ever get lost, you can always `cd` back to the main github folder using an absolute path.

Since my path to this folder is `Documents/GitHub/botany-2025-shell/` I can call

```
cd ~/Documents/GitHub/botany-2025-shell/
```

from any location on my computer's file system, and I will return to the main directory for this course. In this way, an absolute path can be an escape lever for you if you get lost.

[!TIP] You can also call `pwd` on its own to display a full path for any location you find yourself in.

wild cards *

What if we want to `ls` some, but not *all* files in a directory?

In this case we can use `*` as a matching character. From `examplefiles/alexas_section/chloro/`, try:

```
ls -lah
ls -lah *2.txt
ls -lah rps*.txt
```

2. Creating and editing folders and files

`mkdir` (create a folder)

This one is fairly simple! The command makes an empty directory. Just name your directory and you're good to go.

```
mkdir example_dir
cd example_dir
ls
```

`cat` 🐱 (concatenate and print)

`cat` will print out lines of a file, or add to the end of the file. It is most commonly used for the first purpose.

To try it out, navigate to the `/intro_to_CLI_examples` folder.

Now try out `cat` on some files here.

```
cat Firstgenes.fa
```

After typing `cat`, you can press `[Tab]` to activate the autocomplete, which saves you time. You should essentially never be typing file names from memory, as this could result in trying to call a file that does not exist.

`touch` (create a file)

This is an easy way to create files.

```
man touch
mkdir emptyfiles
cd emptyfiles
touch helloworld.txt
ls -lah
cat helloworld.txt #this will not return anything since touch only
creates empty files!
touch file1.txt file2.txt file3.txt
```

Note that you need to add extensions manually by typing them in. The .txt extension is good for right now, other formats carry certain syntax.

>> and > (redirection)

Now that we have our empty files, how do we add to them? The easiest way is to use redirection operators. In the same directory as before:

```
echo "now my file1 has text" >> file1.txt
cat file1.txt
```

What is the difference between >> and >? Try it for yourself:

```
echo "now my file2 has text" >> file2.txt
cat file2.txt
echo "now my file2 has text that is appended" >> file2.txt
cat file2.txt

echo "now my file3 has text" > file3.txt
cat file3.txt
echo "now my file3 has text that is overwritten" > file3.txt
cat file3.txt
cat *.txt
```

> overwrites any text existing in the file, while >> appends your text to the end of the file. When iterating across multiple inputs, it's very useful to write to a file using >>. It's a simple way to generate a summary file, or to store information about what you've run so far.

You can also use this with `cat` to overwrite or append files to each other without printing to the terminal.

```
touch file4.txt
echo "this text comes from file4" > file4.txt
cat file2.txt
cat file4.txt >> file2.txt
cat file2.txt
cat file1.txt
```



```
cat file4.txt > file1.txt
cat file2.txt
```

[!TIP] You can create a **new line** in a .txt file by using `\n`. The backslash `\` indicates special characters and is frequently used for this.

```
touch file5.txt
myfilecontents="this\ncan\nbe\nhard\nto\nread"
echo $myfilecontents > file5.txt
cat file5.txt
```

Knowledge check: what is the difference between `echo file5.txt`, `ls file5.txt`, and `cat file5.txt`? Run each one yourself to check.

mv (move and rename)

Now find the `chloro` folder.

```
cd ../chloro/
man mv
```

`mv` will move and rename files.

Let's make a new folder and move files into it. Feel free to open these folders in your normal file browser (Finder for MacOS users) to confirm that you have moved the file. Once you begin using `mv`, you will hopefully realize how useful it is, especially when combined with the **wildcard character** `*`.

```
ls -lah
mkdir ndh
ls -lah

mv ndhA.txt ndh/
ls -lah ndh/
#the file has moved!
#check that it's not in the main folder
ls -lah
ls -lah ndhA.txt
ls -lah ndh/ndhA.txt
```

You can use `mv` with multiple files at once, essentially performing a simple search and moving the files.

```
ls -lah ndh*.txt
mv ndh*
mv ndh*.txt ndh/
```

```
ls -lah ndh*.txt
ls -lah ndh/
```

Using **mv** to rename files is as simple as "moving" the file to a new location with a different name.

```
cd ndh/
cat ndhA.txt
mv ndhA.txt ndhA_chloro.txt
ls -lah
cat ndhA.txt
cat ndhA_chloro.txt
#this is just to show that the file is the same after it is moved!
```

cp (copy)

You might need to copy files like you would in a file browser. I usually use copying to make a backup or test folder for a script, when you're not sure if your code will work the way you want it to. It's a good idea to make a backup of your files in other ways, but this redundancy can definitely save you some trouble.

Let's make a backup of the **chloro** folder.

```
cd ../..
man cp
cp -R chloro chloro2
#-R option allows you to copy a directory and its subfolders
```

If you just want to copy certain files, you can select them using the ***** wildcard.

```
cd chloro2/
mkdir ps
cp ps*.txt ps # copies all files starting with "ps" and ending with ".txt"
to the new ps directory
```

rm (remove file/directory)

This command will **remove** files. However, it does not remove files by sending them to a "Trash" folder first. File deletion with **rm** is functionally a permanent deletion. Therefore, we need to be extra careful when using it. Let's use it to clean up some files and folders in this duplicated **chloro** folder.

We use the **-i** option here to ensure that we get an extra layer of confirmation before we delete files.

The **-r** option allows us to delete recursively (aka everything within a folder will be deleted along with the folder).

```
rm -ri ps
rm -i ycf*
```

head and tail

Sometimes, a file can be so large that it's unwieldy to use `cat`. Or, occasionally, you'll only want to see the top or bottom of a file. In this case, you can use `head` or `tail`.

```
head Chloroplast_Genes.fa
# to get only the first two lines:
head -n 2 Chloroplast_Genes.fa

tail Chloroplast_Genes.fa
# to get only the last two lines:
tail -n 2 Chloroplast_Genes.fa
```

3. Helpful commands, introduction to programming structures

clear, history

Here are some other useful basic commands.

`clear` is possibly the simplest command you will learn today. It clears your screen. And it's named in a way that makes sense! Nice!

```
clear
```

In the same vein, `history` will simply print the history of the commands you have run.

```
history
#this will show your command history
history -c
#this will clear your history, but you shouldn't need to do this.
```

echo (print a variable)

`echo` is another one that pretty much does what it says on the tin!

```
echo "hello world\!"
```

when run, you should see something like this:

```
alexas_section/chloro [main] » echo "hello world\!"  
  
hello world!
```

`echo` can also be used to print variables that you've stored like so:

```
echo "hello world\!"  
  
my_var="hello everyone\!"  
echo $my_var
```

This is useful for when we don't want to type the same thing multiple times. Setting a variable can be used for **strings**, which are essentially strings of words and are

A short introduction to variables

Variables are a type of data that can be called again after they are declared.

Variable type	Description	Example command
string	a phrase consisting of numbers, letters, spaces, or other characters	<code>my_string="hello everyone\!" ; echo \$my_string</code>
number	an integer or float that can be used in mathematical equations	<code>my_num=4 ; echo \$((myv*2)); my_float=3.5; echo \$((my_float+2))</code>

[!TIP] When writing more than one line of code, you can use an **endline** character (`;`) to indicate a new line. The results of these lines will be the same, try it out!

```
my_var_a="my string is going to print soon!\!" ; echo $my_var
```

```
my_var_b="my string will print after I echo it!\!"  
echo $my_var
```

For loops

When we have a command that we want to call multiple times, for multiple files, we can use a **loop** to complete the same action repeatedly.

This is useful when we have a large amount of files.

Navigate to the `alexas_section/chloro` folder. Each file in this folder represents a gene.

Recaps/cheat sheets

Part 1 recap

Command	Short description of what it does	Common uses/reminders
<code>ls</code>	List files/folders	<code>ls -lah</code> (human readable format with more info)
<code>man</code>	Manual	<code>man ls</code> (show manual for <code>ls</code> command) ; Q to quit
<code>cd</code>	Change directory	<code>cd ..</code> (<code>cd</code> up one folder)
shortcuts	too many to list! They make your life easier	Up/down on keyboard to scroll thru history, tab to autocomplete
file paths	Navigate around relative to current location	<code>.</code> (current directory), <code>..</code> (up one directory)
* (wild cards)	Select multiple files/folders that match a pattern	<code>ls *.txt</code> (list all files which end in <code>.txt</code> in this directory)

Part 2 recap

Command	Short description of what it does	Common uses/reminders
<code>mkdir</code>	Make folders	<code>mkdir exampleDir</code>
<code>touch</code>	Make empty files	<code>touch examplefile.txt</code>
<code>cat</code>	Concatenate (print file contents)	<code>cat Chloroplast_Genes.fa</code>
<code>>, >></code>	Redirects standard output to a file	<code>echo "text" >> file1.txt</code>
<code>mv</code>	Move folder/file	<code>mv (move) fileto move locationto move</code> -- can be used to rename files/folders
<code>cp</code>	Copy folder/file	<code>cp originalfile locationofcopy</code> , <code>-R</code> option makes it folder-compatible
<code>rm</code>	Remove file/folder	<code>rm -i fileto remove</code> , <code>-r</code> makes this folder-compatible
<code>nano</code>	File editor	<code>nano filetoedit.txt</code> , can also be used to create files
<code>head</code>	Print file head	<code>head -n 10 examplefile.txt</code> , <code>-n</code> specifies number of lines, default without option is 5
<code>tail</code>	Print file end	<code>tail -n 12 examplefile.txt</code> , <code>-n</code> specifies number of lines, default without option is 5

Part 3 recap

Command	Short description of what it does	Common uses/reminders
<code>clear</code>	Clear screen	Can also press <code>CTRL+L</code>
<code>history</code>	Show command history	<code>history -3</code> prints 3 past commands, <code>history -c</code> to clear
<code>echo</code>	Print contents of a variable	<code>echo 'helloworld'</code> or a variable: <code>echo \$SHELL</code> , either one you have set or a built-in variable
<code>for loop</code>	Perform actions repeatedly on specific files	<code>for x in item1 item2 item3; do command; second optional command; done; (; is a line separator)</code>

Extras: Problem solving and finding answers online

When looking for help, you can take a few different approaches. Classic Google searches, in many cases, can be difficult to navigate when you are unfamiliar with a new programming language, and the vocabulary for finding your answer might escape you. Sometimes you can find a needle in the haystack by Googling "how to make this thing do another thing" because you lack terminology, but learning how to accurately describe the issue you are facing will go a long way in getting you closer to an answer.

The advice below will help you the most if you are flexible and broad with your search strategy. This is to say that you should open at least the top three hits to a question or error message. Consider the answers on the page and whether they seem to explain your issue in terms that you understand. Since there are many ways to solve a problem, you should find the easiest and simplest way to do it. If your simple variable assignment error can be solved in one line, why do it in ten?

Consider if each approach makes sense to you. I try to avoid helpful tips that include installing packages or modules unless it's my last resort.

If none of the top 3-ish hits seem to solve my problem, I usually change the wording and try again.

One piece of advice is that you should always paste your error message into Google. You will find that many people have had the same issue.

However, if the error message is over 2 lines, and wordy, consider trimming some of it. If the error message includes your own directory structure, remove that unless you're trying to do something with the basic file system. Because every user has their own directory setup, you'll get more specific hits if you prioritize the error in question.

Safety risks while looking for answers online

Please exercise caution when copying and pasting code *that you don't understand* from online sources into your terminal. As a beginner, you should prioritize code that makes sense within your current understanding, and build upon your current knowledge. You may not be working with complex databases from day 1, but that's *totally fine*.

This article runs through some reasons why copying and pasting code can be a security risk:

<https://stackoverflow.blog/2019/11/26/copying-code-from-stack-overflow-you-might-be-spreading-security-vulnerabilities/>

Final notes

Extra materials for the extra-material-inclined

- [ShellCheck \(editor plugin, or online\)](#)
- [Bash cheat-sheet](#)
- [Linux command line textbook](#)

Copyright information: redux

This course material, as well as the accompanying slides and videos are all © Alexa S. Tyszka 2025. These materials may not be distributed or uploaded to any third-party websites without permission of the author.